Process Mining for Object-oriented Processes with Dynamic-temporal Relations

Jost Götte[1], Maximilian Harms[1], and Henrik Leopold[1]

Kühne Logistics University, Hamburg, Germany {firstname.lastname}@klu.org

Abstract. Process mining analyzes and improves business processes using event data. Object-Centric Process Mining (OCPM) extends this by capturing interactions between multiple object types. However, current methods struggle with hierarchical structures where one object contains multiple related objects, called collection objects. This paper formally introduces collection objects within OCPM and identifies two key challenges: multi-level event execution and object lifecycle mismatches. We propose a split-tree mining approach that models dynamic splits in collection objects using hierarchical temporal relations. Our method preserves relationships across object levels and time and, thus, enables accurate performance analysis. We evaluate our approach using a simulated order-to-delivery process with varying object complexity. The results show that our approach outperforms traditional and existing OCPM techniques in metrics such as lead time, average delivery time, and delivery spread. This work establishes a foundation for extending object-centric analysis to processes exhibiting dynamic, hierarchical behaviors, addressing limitations of prior methods and improving process insights in complex domains.

Keywords: OCPM · temporal relations · collection object · split deliveries · KPIs.

1 Introduction

Process mining provides methods for analyzing and improving operational processes using event data. It helps identify deviations, inefficiencies, and improvement opportunities across domains such as finance [22], healthcare [8], and supply chain management [19]. Traditional techniques assume flat event logs with single case identifiers. While effective, this structure poses limitations, particularly when multiple process entities interact within a single execution [3].

Object-centric process mining (OCPM) addresses this by modeling multiple interacting objects within a process. Rather than using a single case notion, OCPM links events to all relevant objects, providing a more accurate and flexible representation of real-world processes [3]. However, most OCPM approaches assume objects refer to singular items, like an invoice, an assumption that often fails in practice.

In many domains, objects represent collections, i.e., aggregated entities containing multiple, potentially heterogeneous items. We refer to these as *collection*

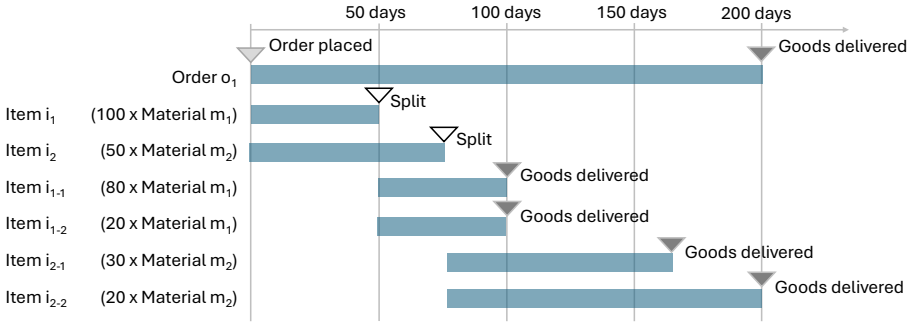

Fig. 1. An order-to-delivery process illustrating the dynamics resulting from collection objects that change over time due to splits.

objects. Traditional mining often treats such relationships with type and quantity attributes in events, but this oversimplifies the inherent complexity. In OCPM, treating collections as atomic units can obscure important behaviors.

To illustrate this, consider the order-to-delivery (O2D) process from a business-to-business (B2B) setting shown in Figure 1[1]. A customer places order o_1 containing two items: item i_1 with 100 units of material m_1, and item i_2 with 50 units of material m_2. Due to shipping delays, the supplier performs partial deliveries for both materials. On an object level, this results in item i_1 being split into i_{1-1} and i_{1-2}, and i_2 into i_{2-1} and i_{2-2}. The first of each pair is delivered earlier; the second, later. Crucially, the order object o_1 is marked *delivered* only once all items are shipped. Analyzing only the order level can thus misrepresent the delivery process. However, focusing on item-level objects poses challenges as some are created dynamically (e.g., i_{2-2} emerges later). Moreover, items are hierarchically and temporally related, analyzing i_{2-2} requires linking it to i_2, i_{2-1}, and order o_1.

This example highlights that existing (object-centric) process mining approaches are not equipped to handle such dynamic object behavior. Two key challenges arise: 1) multi-level event execution and 2) mismatch of object lifecycles. Challenge 1 refers to the fact that collection objects manage multiple lower-level objects. In our example, the order is a collection of items with distinct behaviors, some picked immediately, others split. A meaningful analysis must capture hierarchical relationships, allowing traceability at multiple abstraction levels. Challenge 2 refers to the phenomenon that splitting an item terminates its lifecycle and initiates those of new items. This results in partially overlapping lifecycles. Analyzing performance metrics such as delivery time, hence, requires understanding these temporal relationships across object lifecycles.

In this paper, we argue that collection objects require explicit treatment in object-centric process analysis. We show that behavioral deviations within item groupings are often hidden by standard aggregation, leading to incomplete or

[1] The scenario in Figure 1 is simplified but realistic. Several ERP systems, including SAP and Oracle, implement order splits in this manner.

misleading performance assessments. Using order management as a case study, we explore how collection objects affect performance analysis and propose a method that accounts for their internal variability.

The remainder of the paper is structured as follows: Section 2 presents our method. Section 3 evaluates our approach against traditional and object-centric mining. Section 4 discusses related work, and Section 5 concludes.

2 Analyzing processes with Collection Objects

This section presents our method for analyzing processes with collection objects. Building on the object-centric definitions in [7] and [4], we extend them with collection-object-specific concepts and a corresponding mining algorithm.

We start with the universe of events, denoted U_e. Each event is an instance of an event type from U_{etype} and associated with a timestamp from U_{time}. Events are also linked to objects from the universe U_o, categorized into types from U_{otype}. Each object belongs to exactly one object type. Objects may also relate to other objects. Relationships between events and objects, and among objects themselves, can be qualified by types from U_{qual}.

Definition 1 (Object-centric Event Log). An object-centric event log is a tuple $L = (E, ET, O, OT, \pi_{etype}, \pi_{time}, \pi_{otype}, \pi_{trace}, O2O)$ consisting of:

- a set of events $E \subseteq U_e$,
- a set of event types $ET \subseteq U_{etype}$,
- a set of objects $O \subseteq U_o$,
- a set of object types $OT \subseteq U_{otype}$,
- a function π_{etype}: $E \to ET$, mapping events to their respective event type,
- a function $\pi_{otype} : O \to OT$, mapping objects to their respective object type,
- a function $\pi_{time} : E \to U_{time}$, mapping each event to their timestamp,
- a function $\pi_{trace} : O \to E^*$, mapping each object to a sequence of events such that $\forall_{o \in O} \pi_{trace(o)} = \langle e_1, \ldots, e_n \rangle \wedge \forall_{i \in 1,\ldots,n-1} \pi_{time}(e_1) \leq \pi_{time}(e_{i+1})$.
- a set of qualified object-object-relationships: $O2O \subseteq O \times U_{qual} \times O$.

Object-centric process mining (OCPM) typically assumes that each object $o \in O$ refers to a singular entity (e.g., invoice, patient, order). We argue this is insufficient for many use cases and introduce the concept of a collection object.

Definition 2 (Collection Object). Let $o \in O$ be an object in an event log L per Definition 1. We call o a collection object if it relates to multiple objects $o'_1 ... o'n$ of the same object type: $\pi otype(o'1) = ... = \pi otype(o'_n)$. The types of o and its lower-level objects may differ. This is indicated via a qualifier like "consists of" in $O2O$. The set of all collection objects of L is therefore given by

$$CO = \left\{ o \in O \;\middle|\; \begin{array}{l} \exists o' \in O \setminus \{o\} : (o, \text{"consists of"}, o') \in O2O \wedge \\ \pi_{otype}(o) = \pi_{otype}(o') \wedge \ (o', \text{"consists of"}, o) \notin O2O \wedge \\ \forall o'' \in O, \ (o, \text{"consists of"}, o'') \in O2O \Rightarrow \pi_{otype}(o'') = \pi_{otype}(o') \end{array} \right\}$$

This definition allows us to analyze the process at different levels of abstraction. Consider the order-item relationship from the running example: the process can be examined either at the order level or at the item level. In the former, order objects act as representatives for their lower-level item objects, enabling the aggregation of relevant measures such as quantities or durations. In the latter, each item is analyzed individually, allowing for a more fine-grained perspective.

A key peculiarity of the collection objects considered in this paper is the possibility of a *collection object split*. Again referring to the running example, suppose multiple material units of an order item are ready for shipment. In typical B2B scenarios [21], a supplier might choose to ship the available units immediately rather than wait for the rest. As a result, the original item is split into two: the shipped units and the remaining units, forming two new collection objects.

Definition 3 (Collection Object Split). et $co \in CO$ be a collection object. A split deletes co and creates $n \geq 2$ new collection objects $co_1, \ldots, co_n$ of the same type. This is reflected in the OCEL either via a dedicated "split" event or via a "delete" event for the original object and multiple "create" events for the new ones.

Splits introduce hierarchical temporal relationships. When a split occurs, the original object becomes obsolete and its sub-objects are reassigned. The new collection objects succeed the old one, forming the temporal relation.

Definition 4 (Hierarchical Temporal Split Relation). Let $CO \subseteq O$ be the set of collection objects. We define the split relation $R_{split} \subseteq CO \times U_{qual} \times CO$ such that:

1. $(co_1, qual, co_2) \in R_{split}$ indicates that co_2 is a successor of co_1 in the direction of the deleted (co_1) to the created object (co_2)
2. a collection object cannot be its own successor: $\forall o_1, o_2 \in Oqual \in U_{qual} : o_1 = o_2 \rightarrow (o_1, qual, o_2) \notin R^+_{split}$ with R^+_{split} denoting the transitive closure of R_{split}.

We define $succ : CO \rightarrow \mathcal{P}(CO)$ (the powerset of CO) that obtains all successors of a given collection object, $succ(co) = \{co' \in CO \mid (co, qual, co') \in R^+_{split}\}$. We further use $CO_{roots} = \{co \in CO \mid \nexists\, co' \in CO : (co', qual, co) \in R^+_{split}\}$ to refer to all collection objects that represent roots in the split relation.

As shown, collection objects can succeed each other, potentially desynchronizing object lifecycles (cf. Challenge 2). To mitigate this, analyses of lower-level collection objects must begin at the split predecessors to prevent drawing wrong conclusions.

We therefore propose to mine the split relation, recursively linking an object (e.g., an order) to its leaf-level collection objects (e.g., delivered items). This forms a directed acyclic graph, or *split tree*, where nodes represent collection objects and edges represent split successions. The tree evolves hierarchically over time as splits occur.

Algorithm 1 Split Tree Mining

```
 1: procedure getSplitTrees(OCEL,object)
 2:     split_trees ← NewList()
 3:     roots ← {co ∈ CO_roots | ∃ (object, co) ∈ O2O}
 4:     for each root in roots do
 5:         split_tree ← NewTree()
 6:         queue ← NewQueue()
 7:         Enqueue(queue, root)
 8:         AddNode(split_tree, root)
 9:         while queue ≠ empty do
10:             current ← Dequeue(queue)
11:             if succ(current) ≠ ∅ then
12:                 children ← succ(current)
13:                 AddNodes(split_tree, children)
14:                 Enqueue(queue, children)
15:             else
16:                 AddLeaf(split_tree, current)
17:             end if
18:         end while
19:         Append(split_trees, split_tree)
20:     end for
21:     return split_trees
22: end procedure
```

Algorithm 1 implements a breadth-first search to extract split trees from an OCEL. Starting from the object of interest, we identify roots in $O2O$ (line 2) and for each root, we create a new tree and queue and initialize both with the root (lines 5-8). We repeat the following recursively: take the first element out of the queue and check if it has successor elements using the *succ* function, i.e., children (lines 10-11). If that is the case, we add the children to both, the current split tree and the queue (lines 13-14). Otherwise, it is added to the leaves of the split tree (line 16). Finally, we return all created split trees (line 21).

Split trees allow analysis across object levels (Challenge 1) and mismatching lifecycles (Challenge 2). Algorithm 2 formalizes how to compute KPIs from them. Given an object, a KPI function (e.g., delivery time), and an aggregation method (e.g., average), the algorithm obtains all split trees related to the given object and iterates over the leaves (lines 2-6). It then calls the respective KPI function on the events of the underlying OCEL associated with the leaf object (line 7). Finally, the KPI is aggregated and returned (line 11) depending on the desired level of analysis.

As a concrete example of KPI computation, consider the *weighted delivery time*. In our scenario, this involves calculating the time difference between when the order was placed and when each item (represented by a leaf node in the split tree) was delivered. Similar calculations can be applied to other KPIs. Given the challenges discussed, particularly those related to multi-level execution and

Algorithm 2 KPI Computation based on Split Trees

```
1: procedure ComputeKPI(object, kpi_func, aggregate_func, level)
2:     split_trees ← GetSplitTrees(object)
3:     results ← NewList()
4:     for each tree ∈ split_trees do
5:         tree_results ← NewList()
6:         leaves ← GetLeaves(tree)
7:         for each leaf ∈ leaves do
8:             result ← kpi_func(object, leaf)
9:             Append(tree_results, result)
10:        end for
11:        if level== lower then
12:            Append(results, aggregate_func(tree_results))
13:        else
14:            Append(results, tree_results)
15:        end if
16:    end for
17:    if level== upper then
18:        return aggregate_func(results)
19:    else
20:        return results
21:    end if
22: end procedure
```

lifecycle mismatches, it is essential to rely on the split-tree-based approach when analyzing processes involving collection objects.

3 Evaluation

In this section, we test our split-tree-based method against traditional process mining techniques in an order-to-delivery setting. The goal is to evaluate its effectiveness in addressing the two key challenges outlined in **??**, multi-level event execution and object lifecycle mismatches caused by item splits. We simulate a warehouse that generates object-centric event logs for each order[2]. Subsection 3.1 describes the simulated process. Subsection 3.2 details data generation. Subsection 3.3 presents baseline methods. Subsection 3.4 introduces evaluation metrics, and subsection 3.5 reports the results.

3.1 Simulated Process

Fig. 2 shows the BPMN diagram of the simulated warehouse process, a simplified order-to-delivery workflow capturing partial deliveries and item splits. The process starts with "Place Order", which initiates an order and creates associated

[2] Implementation, datasets, and supplementary materials are available at: https://github.com/MHarmsKlu/SimulationSplittedDeliveries.git.

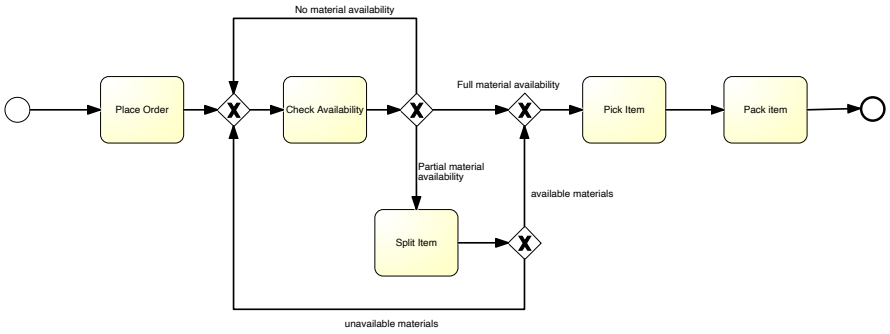

Fig. 2. Overview of the order process we use for our evaluation as BPMN.

item objects, each representing a quantity of a material. The order then follows administrative steps like "Send Invoice" and "Receive Payment".

Item availability is continuously probed via "Check Availability", leading to three paths: 1) *no material availability*: the item is skipped and re-evaluated later, 2) *full material availability*: the item is picked, packed, and shipped, and 3) *partial material availability*: a "Split Item" activity divides the item into two, one for available units and one for the remainder. The available part proceeds, while the remainder loops back for rechecking. Splitting can occur multiple times until all materials are delivered. Each final group proceeding through the full flow becomes a leaf in the split tree (as defined in Section 2).

3.2 Data Generation

We simulate a warehouse environment across ten 1000-day runs, each generating event data for stock replenishment. The simulations reflect increasing levels of deliveries, leading to more item splits. In the first run, all orders are delivered in full. In later runs, the average number of partial deliveries per order increases, simulating scenarios from single shipments to complex multi-shipment processes. Splits per order follow a normal distribution, with a standard deviation of 10% of the mean.

The simulation mimics a realistic order-to-delivery flow, where item availability drives when and how items are split. Each order contains one or more materials with defined quantities and delivery periods. Deliveries are made in parts based on current availability, using probabilistic distributions. Timestamps are generated from random distributions within fixed intervals, yielding varied and realistic delivery schedules.

Each run produces an object-centric event log (OCEL 2.0), linking events to Orders, Items, and Packages. These logs reflect the hierarchical and temporal dependencies introduced by item splits. Packages link to orders indirectly via intermediate item instances, preserving the process's structural complexity.

3.3 Configurations and Baselines

Since our split-tree-based method facilitates an analysis on different levels, we use two configurations:

- **Split-tree-based - order level (STO)**: Aggregates KPIs at the order level using the upper-level configuration of 2. Results from all split trees are combined per order.
- **Split-tree-based - item level (STI)**: Aggregates KPIs at the item level using the lower-level configuration of 2. Each split tree contributes individual item-level KPIs.

We compare these two configurations against three traditional process mining approaches:

- **Case-centric - Order as case (CCO):** Uses orders as cases. Aggregates all related item events to the order-level, ignoring item-level variability and lifecycle mismatches.
- **Case-centric - Item as case (CCI):** Uses items as cases, offering maximum item-level visibility. However, it ignores order-level context and duplicates split events across cases.
- **OCPM + OC PPINOT (OCP):** Applies OC-PPINOT [10] on OCEL logs. It uses traceability functions to follow object lifecycles through a single level of splits. However, it cannot fully trace all delivery events when multiple splits occur, which limits its effectiveness in processes with deep and variable hierarchical dynamics.

.

For evaluation, OCPM approaches are provided with OCEL 2.0 logs in JSON format, while traditional case-centric methods use CSV-based flat event logs generated through the described flattening procedure. All resulting KPI values are exported in tabular form as Excel files (XLSX).

3.4 Metrics

In our evaluation, we focus on three key metrics to assess both process performance and model quality:

- **Lead time**: Duration from order placement to final delivery, reflecting overall process efficiency from a customer perspective [13]. It provides a high-level view of the order duration and is essential for assessing overall process efficiency from the customer's perspective.
- **Average delivery time**: Mean duration from order placement to each individual delivery. This metric captures responsiveness in multi-shipment scenarios.
- **Delivery spread**: Time between the first and final shipment, indicating the temporal dispersion of partial deliveries.

These metrics collectively evaluate how well each method addresses the challenges posed by collection objects and item splits.

3.5 Results

The results of our evaluation are presented in Table 1. Across all metrics, our split-tree-based method consistently outperforms both traditional and baseline object-centric approaches, particularly under increasing process complexity caused by item splits.

AoD	Lead Time					Average Delivery Time					Delivery Spread				
	CCO	CCI	STO	STI	OCP	CCO	CCI	STO	STI	OCP	CCO	CCI	STO	STI	OCP
1	**9.71**	**9.72**	**9.71**	**9.72**	-	**9.71**	**9.72**	**9.71**	**9.72**	-	**0**	**0**	**0**	**0**	-
2	**12.76**	7.63	**12.76**	**12.44**	10	12.76	7.63	**11.13**	**11.02**	8	0	0	**3.02**	**2.62**	3
3	**17.41**	6.28	**17.41**	**16.98**	12	17.41	6.28	**13.57**	**13.34**	9.50	0	0	**7.24**	**6.81**	5
4	**21.84**	5.69	**21.84**	**21.98**	6	21.84	5.69	**15.66**	**15.72**	6	0	0	**12.19**	**12.34**	0
5	**26.76**	5.44	**26.76**	**26.51**	11	26.76	5.44	**18.28**	**18.26**	11	0	0	**16.58**	**16.24**	0
6	**31.77**	5.33	**31.77**	**31.85**	13	31.77	5.33	**20.70**	**20.77**	13	0	0	**21.54**	**21.55**	0
7	**36.61**	5.16	**36.61**	**36.63**	8	36.61	5.16	**22.91**	**22.92**	8	0	0	**26.76**	**26.80**	0
8	**41.60**	4.93	**41.60**	**40.95**	12	41.60	4.93	**25.66**	**25.34**	12	0	0	**31.62**	**30.86**	0
9	**45.72**	4.98	**45.72**	**45.32**	11	45.72	4.98	**27.77**	**27.49**	11	0	0	**35.63**	**35.28**	0
10	**50.85**	4.92	**50.85**	**50.62**	7	50.85	4.92	**30.11**	**29.95**	7	0	0	**40.76**	**40.49**	0

Table 1. Comparison of lead time, average delivery time, and delivery spread across different approaches as the average amount of deliveries (AoD) increases. Correct values are in bold.

As for the *lead time*, traditional methods like CCI provide accurate lead times only in single-shipment scenarios. With increasing splits, they overlook intermediate deliveries and underestimate durations. In contrast, our split-tree methods (STO, STI) maintain accurate lead time estimates by correctly tracing hierarchical object lifecycles and dependencies. As for the *average delivery time*, CCI partially captures the delivery dynamics but underestimates the durations due to truncated life cycles, ignoring links to the original order. Split-tree methods accurately relate each delivery to the initial order placement, yielding reliable averages across all scenarios. As for the *delivery spread*, only the split-tree methods compute delivery spread correctly. CCO reports zero spread, and CCI consistently underestimates it. Our method captures the full temporal range of deliveries by traversing the entire split hierarchy, essential for reflecting real-world delivery dispersion.

OCP is limited by its fixed-depth traceability. It misses deeper split levels, intermediate deliveries, and direct shipments, leading to incomplete results and declining accuracy as the number of splits increases. Our approach remains stable and accurate across all evaluated scenarios, regardless of the number of deliveries or splits. Competing methods degrade with complexity, demonstrating the scalability and robustness of split-tree-based analysis in hierarchical, object-centric contexts.

These results confirm our hypothesis: conventional methods are inadequate for processes with multi-level object relations and lifecycle mismatches. Our split-tree-based approach effectively addresses both challenges, multi-level execution and object lifecycle mismatch, hence, offering accurate, scalable, and meaningful

performance insights, particularly for complex domains such as supply chain management.

4 Related Work

This section positions our contribution in the context of existing research. We organize the discussion into three areas: 1) OCPM foundations and extensions, 2) performance analysis in OCPM , and 3) the use of process mining in supply chain management.

OCPM Foundations and Extensions. OCPM enhances process modeling by associating events with multiple interacting objects. Foundational concepts like OCEL 2.0 [11] and object-centric directly-follows graphs (OCDFG) [2] enable to model, discover, and analyze processes beyond the single-case views. Advanced models include object-centric Petri nets [1] and object continuation graphs [5], which relate to our split-tree approach. Other works address timing dependencies [18] and quantity-aware mining [12] but do not fully capture hierarchical temporal dynamics or integrate collection objects as we do.

Performance Analysis in OCPM. Initial performance techniques focused on event- or activity-level metrics, while recent work extends these to the object-centric context. For example, Estrada-Torres et al. [10] introduced traceability functions to group related objects but lack recursion needed for variable-depth split-trees. Similar limitations appear in process querying approaches [17]. Park et al.[20] proposed OPerA to generalize classical metrics in an object-centric setting, focusing on object collaboration rather than domain-specific KPIs. Other advanced methods include performance spectrums [9] and statistical inference from object creation graphs [6]. These generally assume static object relationships. Our work extends these by introducing hierarchical temporal splits and split-tree-based KPI computation to capture dynamic object lifecycles.

Process Mining in Supply Chain Management. Several studies have explored process mining in supply chains. Jokonowo et al. [16] reviewed 21 papers mostly on process discovery, with many validated by case studies. Jacobi et al. [14] identified applications in model construction, providing alerts and decision support, and automated adjustments. However, these primarily rely on traditional paradigms. Few address object-centric process mining for complex supply chains, a key focus of our work. Schukat et al. [21] highlight that traditional methods often misinterpret split deliveries in Order-to-Cash processes as noise, obscuring process logic. Jans et al. [15] proposed aggregation to unify split events, but this risks masking important behavior that our approach preserves and analyzes.

5 Conclusion

In this paper, we introduced a novel process mining approach for collection objects, compound entities that can undergo dynamic structural changes like splits. These behaviors pose two key challenges for existing methods: multi-level

event execution and object lifecycle mismatch. We formally defined collection objects within the OCPM framework and proposed a split-tree-based method to address these challenges.

Our approach captures the hierarchical and temporal dynamics of object splits, enabling more accurate and robust performance analysis at both collection and item levels. Using a controlled simulation of an order-to-delivery process with varying degrees of splits, we showed that traditional and baseline OCPM methods struggle as complexity grows, while our method consistently delivers accurate results across multiple metrics.

Although promising, future work must validate our method on real-world datasets and explore its applicability to other process mining tasks like conformance checking and predictive analytics. Additionally, the current method assumes only split-based evolution and does not yet handle scenarios where collection objects grow, consolidate, or involve KPI events occurring between splits. Addressing these limitations will improve the generalizability and practical impact of our approach across diverse domains.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aalst, W., Berti, A.: Discovering object-centric petri nets (10 2020)
2. van der Aalst, W.M.P.: Object-centric process mining: Dealing with divergence and convergence in event data. In: Ölveczky, P.C., Salaün, G. (eds.) Software Engineering and Formal Methods. pp. 3–25. Springer International Publishing, Cham (2019)
3. van der Aalst, W.M.: Object-centric process mining: dealing with divergence and convergence in event data. In: Software Engineering and Formal Methods: 17th International Conference, SEFM 2019, Oslo, Norway, September 18–20, 2019, Proceedings 17. pp. 3–25. Springer (2019)
4. Adams, J.N., Park, G., van der Aalst, W.M.: Preserving complex object-centric graph structures to improve machine learning tasks in process mining. Engineering Applications of Artificial Intelligence **125**, 106764 (2023)
5. Berti, A., Herforth, J., Qafari, M.S., van der Aalst, W.M.P.: Graph-based feature extraction on object-centric event logs. International Journal of Data Science and Analytics **18**(2), 139–155 (Aug 2024)
6. Berti, A., Jessen, U., Park, G., Rafiei, M., Aalst, W.: Analyzing inter-connected processes: Using object-centric process mining to analyze procurement processes (04 2023)
7. Berti, A., Koren, I., Adams, J.N., Park, G., Knopp, B., Graves, N., Rafiei, M., Liß, L., Unterberg, L.T.G., Zhang, Y., et al.: Ocel (object-centric event log) 2.0 specification. arXiv preprint arXiv:2403.01975 (2024)
8. De Roock, E., Martin, N.: Process mining in healthcare–an updated perspective on the state of the art. Journal of biomedical informatics **127**, 103995 (2022)
9. Denisov, V., Fahland, D., van der Aalst, W.M.P.: Unbiased, fine-grained description of processes performance from event data. In: Weske, M., Montali, M., Weber,

I., vom Brocke, J. (eds.) Business Process Management. pp. 139–157. Springer International Publishing, Cham (2018)
10. Estrada-Torres, B., del Río-Ortega, A., Resinas, M.: Defining process performance measures in an object-centric context. In: Cabanillas, C., Garmann-Johnsen, N.F., Koschmider, A. (eds.) Business Process Management Workshops. pp. 210–222. Springer International Publishing, Cham (2023)
11. Ghahfarokhi, A.F., Park, G., Berti, A., van der Aalst, W.M.P.: Ocel: A standard for object-centric event logs. In: Bellatreche, L., Dumas, M., Karras, P., Matulevičius, R., Awad, A., Weidlich, M., Ivanović, M., Hartig, O. (eds.) New Trends in Database and Information Systems. pp. 169–175. Springer International Publishing, Cham (2021)
12. Graves, N., Koren, I., Rafiei, M., van der Aalst, W.M.P.: From identities to quantities: Introducing items and decoupling points to object-centric process mining. In: De Smedt, J., Soffer, P. (eds.) Process Mining Workshops. pp. 462–474. Springer Nature Switzerland, Cham (2024)
13. Gunasekaran, A., Patel, C., Tirtiroglu, E.: Performance measures and metrics in a supply chain environment. International journal of operations & production Management **21**(1/2), 71–87 (2001)
14. Jacobi, C., Meier, M., Herborn, L., Furmans, K.: Maturity model for applying process mining in supply chains: Literature overview and practical implications. Landscape Journal **2020**, 1 (2020)
15. Jans, M.: From relational database to valuable event logs for process mining purposes: a procedure (01 2017)
16. Jokonowo, B., Claes, J., Sarno, R., Rochimah, S.: Process mining in supply chains: A systematic literature review (01 2018)
17. Küsters, A., van der Aalst, W.M.P.: Ocpq: Object-centric process querying and constraints. In: Research Challenges in Information Science. pp. 383–400. Springer Nature Switzerland, Cham (2025)
18. Liss, L., Adams, J.N., van der Aalst, W.M.P.: Totem: Temporal object type model for object-centric process mining. In: Marrella, A., Resinas, M., Jans, M., Rosemann, M. (eds.) Business Process Management Forum. pp. 107–123. Springer Nature Switzerland, Cham (2024)
19. Oldenburg, F., Hoberg, K., Leopold, H.: Process mining in supply chain management: state-of-the-art, use cases and research outlook. International Journal of Production Research **63**(8), 2889–2904 (2025)
20. Park, G., Adams, J.N., Aalst, W.: OPerA: Object-Centric Performance Analysis, pp. 281–292 (10 2022)
21. Schukat, G.: Schukat: Process Mining Enables Schukat Electronic to Reinvent Itself, pp. 135–142. Springer International Publishing, Cham (2020)
22. Werner, M., Wiese, M., Maas, A.: Embedding process mining into financial statement audits. International Journal of Accounting Information Systems **41**, 100514 (2021)